

Hashing Algorithms Assignment

Objective:

To understand and implement hashing algorithms and caching algorithms.

LO's achieved

On successful completion of this module the learner will be able to:

1. Discriminate between, and experiment with, different algorithm types such as greedy algorithms, divide-and-conquer, dynamic programming algorithms.
2. ~~Appraise, develop and analyse fundamental graph algorithms including graph search, shortest path, and minimum spanning trees.~~
3. ~~Investigate and experiment with mechanisms for tree storage and balancing.~~
4. Identify requirements on, and implications for, data structures that must support concurrency and/or parallelism; illustrate how those requirements are met

1. Hashing Fundamentals (5 Marks)

Define the following terms and concepts:

- 1.1. Hash Function
- 1.2. Hash Table
- 1.3. Collision in a hash table
- 1.4. Load Factor
- 1.5. Open addressing

2. Compare Hashing with Tree-based Structures: (5 Marks)

- 2.1. Discuss how hash-based data structures (like hash tables) differ from tree-based data structures (like red-black trees or AVL trees).
- 2.2. What are the advantages and disadvantages of each in terms of time complexity for operations such as insert, delete, and search?

3. Hashing Algorithms with Divide and Conquer Approach: (7 Marks)

Implement a hash table using a divide and conquer strategy for handling collisions and resizing.

- 3.1. Create a hash table with open addressing with linear probing for collision resolution.
- 3.2. Apply a divide and conquer approach for rehashing during resizing
- 3.3. Create the following operations using the base code provided:
 - 3.3.1. Insert a key-value pair into the table
 - 3.3.2. Search for a key in the table

- 3.3.3. Delete a key from the table
- 3.3.4. Resize the table when the load factor exceeds 75% of the capacity using a divide and conquer approach for rehashing

4. Hash Table as a Caching Algorithm for Least Recently Used (LRU) Caching. (8 Marks)

Implement a hash table that acts as a cache using the least recently used caching strategy. The table should store key-value pairs and have a fixed capacity. The LRU element should be deleted to make room for any new element.

- 4.1. Create a hash table with 20 values.
 - 4.1.1. A doubly linked list is used to maintain the order of elements
 - 4.1.2. A hash map is used to store the keys and their nodes
- 4.2. If another element is added beyond the cache size, evict the LRU item
- 4.3. Create the following operations using the base code provided:
 - 4.3.1. Insert a key-value pair into the cache
 - 4.3.2. Search for a key in the table, if it exists mark the pair as the most recently used
 - 4.3.3. Delete a key from the table
 - 4.3.4. Evict item when the cache is full

5. Briefly describe how Question 4 (LRU cache) can be updated to support concurrency. (5 Marks)

Consider thread safety, locking mechanisms, and concurrent data structures.

Skeleton Code 1:

```
import java.util.Arrays;

public class HashTableDivideAndConquer {
    private String[] table;
    private int capacity;
    private int size;

    // Constructor to initialize the hash table with a given capacity
    public HashTableDivideAndConquer(int capacity) {
        <<fill in this method>>
    }

    // Simple hash function to map a string key to an index
    private int hash(String key) {
        <<fill in this method>>
    }

    // Insert a key into the hash table
    public void insert(String key) {
        <<fill in this method>>
    }

    // Search for a key in the hash table
    public boolean search(String key) {
        <<fill in this method>>
    }

    // Delete a key from the hash table
    public void delete(String key) {
        <<fill in this method>>
    }

    // Divide and Conquer Resize: Rehash the table by dividing the task into smaller chunks
    private void resize() {
        <<fill in this method>>
    }

    // Rehash remaining elements to fill gaps after a deletion (Divide and Conquer Approach)
    private void rehash() {
        <<fill in this method>>
    }

    // Print the hash table for debugging
    public void printTable() {
        <<fill in this method>>
    }

    public static void main(String[] args) {
        HashTableDivideAndConquer unihashTable = new HashTableDivideAndConquer(20);
```

```
// Insert keys
unihashTable.insert("ATU Letterkenny");
unihashTable.insert("ATU Killybegs");
unihashTable.insert("ATU Sligo");
unihashTable.insert("ATU Galway Mayo");
unihashTable.insert("ATU Killybegs"); //add 15 more sample university campuses


// Print the hash table
unihashTable.printTable();


// Search for a key
System.out.println("Is 'ATU Sligo' in the table? " + unihashTable.search("ATU Sligo")); // true
System.out.println("Is 'ATU Dundalk' in the table? " + unihashTable.search("ATU Dundalk"));
// false


// Delete a key
unihashTable.delete("ATU Galway Mayo");
unihashTable.printTable();
}
}
*****
```

Skeleton Code 2

```
import java.util.HashMap;

class LRUCache {
    // Node class for doubly linked list
    class Node {
        int key, value;
        Node prev, next;

        Node(int key, int value) {
            this.key = key;
            this.value = value;
        }
    }

    private HashMap<Integer, Node> cache;
    private int capacity, size;
    private Node head, tail;

    // Constructor
    public LRUCache(int capacity) {
        <<fill in this method>>
    }

    // Move a node to the front (most recently used)
    private void moveToFront(Node node) {
        <<fill in this method>>
    }

    // Add a node to the front of the list
    private void addToFront(Node node) {
        <<fill in this method>>
    }

    // Remove a node from the list
    private void remove(Node node) {
        <<fill in this method>>
    }

    // Get the value of a key if it exists in the cache
    public int get(int key) {
        <<fill in this method>>
        return -1; // Key not found
    }

    // Insert a key-value pair into the cache
    public void put(int key, int value) {
        <<fill in this method>>
    }

    // Print the cache for debugging
```

```

public void printCache() {
    <<fill in this method>>
}

// Main method to test the LRU Cache implementation
public static void main(String[] args) {
    LRUCache lruCache = new LRUCache(3); // Cache capacity of 3

    // Insert items
    lruCache.put(1, 102345);
    lruCache.put(2, 102342);
    lruCache.put(3, 102303);
    lruCache.printCache();

    // Access key 2 (this will make key 2 the most recently used)
    System.out.println("Get 2: " + lruCache.get(2)); // Should return 2
    lruCache.printCache();

    // Insert a new key, which will evict key 1 (the LRU)
    lruCache.put(4, 105444);
    lruCache.printCache();

    // Access key 3
    System.out.println("Get 3: " + lruCache.get(3));
    lruCache.printCache();

    // Insert another new key, which will evict key 2 (the LRU)
    lruCache.put(5, 103455);
    lruCache.printCache();
}
}

```